UNIDAD 01: EVALUACIÓN DE PRODUCTO `04/11/2026`

OBJETIVO

Diseñar, implementar y evaluar una aplicación que integre los principales conceptos teóricos y prácticos estudiados durante la primera unidad de la asignatura de Sistemas Operativos, mediante la programación de procesos, hilos, mecanismos de sincronización y técnicas de gestión de memoria.

El proyecto permitirá demostrar la comprensión del funcionamiento interno de un sistema operativo, la capacidad para resolver problemas de programación de sistemas y la aplicación de mecanismos de administración de recursos.

DESCRIPCIÓN GENERAL DEL PROYECTO

Cada equipo desarrollará una aplicación de consola denominada **MiniSO**, que permita ejecutar y observar determinadas operaciones relacionadas con la administración de procesos, la concurrencia y la gestión de memoria.

La aplicación deberá organizarse en tres módulos principales:

- **Gestión de procesos**: creación, ejecución y finalización de procesos utilizando llamadas al sistema de Linux.
- **Gestión de hilos y concurrencia**: ejecución concurrente de tareas, sincronización y protección de recursos compartidos.
- **Gestión de memoria**: simulación de asignación de memoria mediante particionamiento y paginación.

Los módulos deberán estar integrados en una única aplicación y ser accesibles mediante un menú principal o una interfaz de comandos.

Se deberá distinguir claramente entre las operaciones realizadas sobre procesos e hilos reales de Linux y las operaciones de gestión de memoria que se implementarán mediante simulaciones.

REQUERIMIENTOS FUNCIONALES

Módulo I: Gestión de procesos

Desarrollar un módulo que permita crear, ejecutar y finalizar procesos mediante las llamadas al sistema correspondientes.

Requerimientos mínimos:

- Crear procesos hijos utilizando `fork()`.
- Ejecutar programas externos mediante alguna función de la familia `exec()`.
- Permitir la ejecución concurrente de al menos dos procesos hijos.
- Obtener y mostrar los identificadores PID y PPID de los procesos creados.
- Permitir que el proceso padre espere la finalización de sus hijos mediante `wait()` o `waitpid()`.
- Mostrar el estado de finalización de cada proceso.
- Controlar los posibles errores de creación y ejecución de procesos.

Evidencia práctica requerida: Ejecutar la aplicación y demostrar la creación de procesos, su relación jerárquica y su finalización.

Utilizar herramientas del sistema operativo, como `ps` o `pstree`, para contrastar los procesos creados por la aplicación con los que administra Linux.

Análisis teórico: Explicar la diferencia entre proceso y programa, el funcionamiento de `fork()`, la creación de procesos hijos, los cambios de estado y el papel del sistema operativo en su administración.

Módulo II: Gestión de hilos y concurrencia

Implementar un escenario de ejecución concurrente utilizando hilos que compartan recursos dentro de un mismo proceso.

Requerimientos mínimos:

- Crear y ejecutar múltiples hilos utilizando POSIX Threads (`pthread`) o los mecanismos equivalentes de C++.
- Implementar el problema productor-consumidor con un búfer compartido de capacidad limitada.
- Utilizar al menos dos hilos productores y dos consumidores.
- Implementar exclusión mutua utilizando mutex.
- Utilizar variables de condición para coordinar a los hilos cuando el búfer esté vacío o lleno.
- Mostrar mediante mensajes las operaciones de producción, consumo, espera y finalización.
- Garantizar que todos los elementos producidos sean consumidos correctamente, sin pérdidas ni duplicaciones.

Evidencia práctica requerida: Demostrar la ejecución concurrente de los hilos y el funcionamiento de los mecanismos de sincronización.

Realizar pruebas modificando el número de elementos producidos y la capacidad del búfer.

Análisis teórico: Explicar las diferencias entre procesos e hilos, los problemas de condición de carrera, exclusión mutua, interbloqueo e inanición.

Además, presentar un escenario hipotético de interbloqueo e indicar cómo podría prevenirse. No será necesario provocar un interbloqueo real en la aplicación.

Módulo III: Gestión de memoria

Desarrollar un simulador que permita visualizar el funcionamiento de dos mecanismos de administración de memoria estudiados en la primera unidad.

A. Particionamiento de memoria

- Definir una cantidad configurable de memoria total.
- Implementar un mecanismo de asignación contigua utilizando el algoritmo *First-Fit*.
- Permitir solicitar y liberar bloques de memoria.
- Mostrar el estado de los bloques libres y ocupados.
- Identificar situaciones de fragmentación externa.
- Permitir la reutilización de espacios liberados y la unión de bloques libres contiguos.

B. Paginación

- Definir un tamaño de página y una cantidad de marcos de página.
- Simular la asignación de páginas de un proceso a marcos de memoria física.
- Construir y mostrar una tabla de páginas.
- Permitir introducir una dirección lógica y obtener su correspondiente dirección física cuando la página esté asignada.
- Identificar direcciones inválidas o páginas no asignadas.

Evidencia práctica requerida: Presentar al menos dos escenarios de asignación y liberación de memoria que permitan observar la fragmentación.

Demostrar la traducción de direcciones lógicas a físicas utilizando una tabla de páginas, contrastando al menos tres resultados con cálculos realizados manualmente.

Análisis teórico: Explicar las diferencias entre particionamiento y paginación, sus ventajas y limitaciones, así como los problemas de fragmentación que pueden presentarse.

REQUERIMIENTOS GENERALES DE IMPLEMENTACIÓN

La aplicación deberá cumplir lo siguiente:

- Desarrollarse en C o C++, utilizando los mecanismos de programación de sistemas estudiados en clase.
- Ejecutarse correctamente en Linux.
- Organizar el código fuente en funciones y archivos de acuerdo con las responsabilidades de cada módulo.
- Presentar un menú que permita acceder a los módulos implementados.
- Permitir que los principales parámetros de las simulaciones sean modificados sin necesidad de alterar la lógica de los algoritmos.
- Mostrar resultados comprensibles para facilitar su análisis.
- Contemplar errores y situaciones excepcionales relevantes.
- Incluir instrucciones para la compilación y ejecución.

No se permitirá sustituir las llamadas al sistema requeridas por simples impresiones que simulen la creación de procesos o hilos.

ENTREGABLES Y EVIDENCIAS DEL DESARROLLO

Con el propósito de evaluar tanto el producto final como el proceso seguido para desarrollarlo, cada equipo deberá presentar los siguientes entregables.

Entregable 1. Código fuente y aplicación funcional

Presentar el proyecto completo en GitHub, incluyendo todos los archivos necesarios para su compilación y ejecución.
Se deberá adjuntar un archivo `README.md` que indique los requisitos, las instrucciones de compilación, la estructura del proyecto y la utilización de la aplicación.

Entregable 2. Informe técnico

Elaborar un informe en Documentos de Google, con una extensión referencial de 6 a 10 páginas, sin contar anexos, que contenga:

1. **Introducción:** finalidad del proyecto y relación con los contenidos de la unidad.
2. **Diseño de la solución:** arquitectura de la aplicación, módulos, diagramas y descripción de los principales algoritmos.
3. **Implementación:** explicación de las llamadas al sistema, estructuras de datos y mecanismos de sincronización utilizados.
4. **Pruebas y resultados:** casos ejecutados, entradas, salidas esperadas, resultados obtenidos y análisis.
5. **Dificultades encontradas:** problemas reales surgidos durante la implementación y procedimientos utilizados para resolverlos.
6. **Conclusiones:** aprendizajes obtenidos y limitaciones de la solución.
7. **Referencias bibliográficas:** fuentes técnicas y académicas consultadas.

El informe deberá privilegiar la explicación de las decisiones de diseño y el análisis de los resultados, evitando descripciones teóricas extensas que no guarden relación directa con el proyecto.

Entregable 3. Registro progresivo del desarrollo

Cada equipo deberá utilizar Git para registrar la evolución del proyecto.
Se exigirá evidencia de, al menos, tres etapas de trabajo:

Etapa	Evidencia mínima
1. Diseño e implementación inicial	Estructura del proyecto, diseño de módulos y primeras funcionalidades
2. Desarrollo e integración	Código de los módulos, pruebas parciales y correcciones
3. Validación final	Integración completa, solución de errores y pruebas definitivas

No será suficiente presentar varios commits creados únicamente al finalizar el proyecto para simular un desarrollo progresivo.

Entregable 4. Evidencias de ejecución y pruebas

Presentar capturas de pantalla y registros de ejecución obtenidos directamente en el entorno Linux. Las evidencias deberán incluir:

- Compilación satisfactoria de la aplicación.
- Creación y finalización de procesos con sus respectivos PID y PPID.
- Ejecución y sincronización de hilos.
- Funcionamiento del búfer compartido.
- Asignación y liberación de particiones de memoria.
- Construcción de tablas de páginas y traducción de direcciones.
- Pruebas de situaciones excepcionales y corrección de errores.

Cada evidencia deberá estar identificada e incluir una breve explicación del resultado observado.
Las capturas aisladas no reemplazarán las demostraciones de funcionamiento.

Entregable 5. Video de demostración

Presentar un video de una duración máxima de **8 minutos**, en el que se evidencie:

1. Compilación de la aplicación desde su código fuente.
2. Ejecución de los tres módulos.
3. Modificación de parámetros de entrada y observación de los resultados.
4. Explicación breve de los principales fragmentos de código.
5. Presentación de un problema de implementación encontrado y de su solución.

El video deberá mostrar directamente el entorno de trabajo y la aplicación en funcionamiento. No será suficiente presentar diapositivas, imágenes o una explicación conceptual.

Deberán participar todos los integrantes del equipo, explicando las partes en cuya implementación hayan intervenido.

SUSTENTACIÓN Y VERIFICACIÓN INDIVIDUAL

La sustentación será obligatoria y tendrá dos componentes:

A. Demostración grupal

Cada equipo presentará su aplicación y demostrará sus principales funcionalidades.

El docente podrá solicitar la ejecución de diferentes escenarios, así como la explicación de los resultados obtenidos.

B. Verificación individual

Cada estudiante deberá demostrar que comprende el código y los conceptos utilizados en el proyecto.

Durante la sustentación, el docente podrá solicitar individualmente:

- Explicar el funcionamiento de una función o fragmento de código seleccionado.
- Predecir el resultado de una ejecución antes de realizarla.
- Identificar la causa de un error introducido o detectado durante las pruebas.
- Modificar un parámetro o una parte del programa y volver a compilarlo.
- Explicar qué sucedería si se elimina un mecanismo de sincronización.
- Resolver manualmente un caso de asignación de memoria o traducción de direcciones y contrastarlo con el simulador.

Las modificaciones solicitadas serán acotadas y estarán relacionadas con las funcionalidades desarrolladas. No se exigirá implementar un módulo nuevo durante la sustentación.

CONSIDERACIONES FINALES

- Todos los integrantes deberán participar en el desarrollo del proyecto y conocer el funcionamiento general de los tres módulos.
- Los proyectos deberán ser desarrollados y probados en un entorno Linux.
- Las bibliotecas y componentes externos utilizados deberán identificarse y justificarse.
- Los equipos deberán contar con el código fuente y los recursos necesarios para ejecutar el proyecto durante su sustentación.
- La entrega de una aplicación funcional no garantiza la máxima calificación si los integrantes no demuestran conocimiento de su diseño, implementación y funcionamiento.